

⊗ This Is Not The Course Website You Are Looking For

This course website is from a previous semester. If you are currently in the class, please make sure you are viewing the latest course website instead of this old one.

</> Homework 03: Weathering with Zipcodes

The goal of this homework assignment is to allow you to practice using regular expressions and filters in shell scripts. In this assignment, you will build on your knowledge of the bourne shell language to write scripts that slice and dice data.

For this assignment, record your scripts and any responses to the following activities in the `homework03` folder of your **assignments** GitLab repository and push your work by **11:59 AM Saturday, February 8**.

Activity 0: Preparation

Before starting this homework assignment, you should first perform a `git pull` to retrieve any changes in your remote GitLab repository:

```
$ cd path/to/repository           # Go to assignments repository
$ git checkout master             # Make sure we are in master branch
$ git pull --rebase               # Get any remote changes not present locally
```

Next, create a new branch for this assignment:

```
$ git checkout -b homework03      # Create homework03 branch and check it out
```

Once this is done, download the `Makefile` and test scripts:

```
# Go to homework03 folder
$ cd homework03

# Download the Makefile
$ curl -LO https://gitlab.com/nd-cse-20289-sp20/cse-20289-sp20-assignments/raw/master

# Add and commit Makefile
$ git add Makefile
$ git commit -m "homework03: add Makefile"

# Download the test scripts
$ make test-scripts
```

Note, you do not need to add and commit the test scripts since the `Makefile` will automatically download them again whenever you run `make`.

You are now ready to work on the activities below.

</> Cause we never go out of style

Note, the provided test scripts are used to verify the correctness of your code (ie. does it do the right thing). Good code, however, requires more than just correct behavior; it demands **consistent formatting** and **concise** implementation. Because of this, your scripts will also be graded on **coding style**. In general, we expect you to strive for **clean code** and will look for the following:

- **Consistency**
 - Naming, whitespace, and indentation
 - No dead code
- **Readability**
 - Idiomatic usage of language syntax and features
 - Code is mostly self-documenting (with comments when necessary)
 - Not too dense, not too sparse
- **Organization**
 - Reasonably sized functions
 - Reasonably structured and ordered

For each of the activities below, 0.5 points are reserved for **coding style**.

Activity 1: Weather (4 Points)

Robert is not amused by all the cold snow on campus and longs for the warmth and comfort of spring. Obsessed with the weather, he compulsively checks weather.gov, which is a website by the National Weather Service. While checking the weather for the fifth time on his phone during class, he realized that he could actually apply some of the things he is learning in class to his current obsession.

For example, he realized that he can fetch the weather information for any zip code, using the following URL:

```
$ curl -sL https://forecast.weather.gov/zipcity.php?inputstring=$ZIPCODE
```

For instance, the weather information for Notre Dame, Indiana can be retrieved using:

```
$ curl -sL https://forecast.weather.gov/zipcity.php?inputstring=46556
```

Looking at the data retrieved from the website, he notices that the forecast and temperature information is wrapped in HTML fields that contain the text "myforecast":

```
...
<p class="myforecast-current">Overcast</p>
<p class="myforecast-current-lrg">37&deg;F</p>
<p class="myforecast-current-sm">3&deg;C</p>
...
```

Robert wants to make a script `weather.sh` that will fetch the weather information for any specified zip code and extract the temperature and forecast. Unfortunately, his phone is buzzing, so he has to step out take take the call...

You decide that you could use some practice with shell scripting and regular expressions, so you help him complete the script.

Task 1: weather.sh

The `weather.sh` script takes three possible flags:

```
$ ./weather.sh -h
Usage: weather.sh [zipcode]

-c    Use Celsius degrees instead of Fahrenheit for temperature
-f    Display forecast text

If zipcode is not provided, then it defaults to 46556.
```

The `-c` flag will have the script output the temperature as Celsius rather than Fahrenheit (the default).

The `-f` flag will have the script output the forecast text before the current temperature.

The `-h` flag will display the usage message and exit with success.

Here are some examples of `weather.sh` in action:

```
# Show current temperature for Notre Dame, Indiana (46556)
$ ./weather.sh
Temperature: 34 degrees

# Show forecast and current temperature for Orange, California (92867)
$ ./weather.sh -f 92867
Forecast:    NA
Temperature: 50 degrees

# Show forecast and current temperature (celsius) for Eau Claire, Wisconsin (54701)
$ ./weather.sh -c -f 54701
Forecast:    Fog/Mist
Temperature: -2 degrees
```

Hints

Here is a skeleton you can use to start your `weather.sh` script:

```
#!/bin/sh

# Globals

URL="https://forecast.weather.gov/zipcity.php"
ZIPCODE=46556
FORECAST=0
CELSIUS=0

# Functions

usage() {
    cat 1>&2 <<EOF
Usage: $(basename $0) [zipcode]

-c    Use Celsius degrees instead of Fahrenheit for temperature
-f    Display forecast text

If zipcode is not provided, then it defaults to $ZIPCODE.
EOF
    exit $1
}

weather_information() {
    # Fetch weather information from URL based on ZIPCODE
}

temperature() {
    # Extract temperature information from weather source
    weather_information | ...
}

forecast() {
    # Extract forecast information from weather source
    weather_information | ...
}

# Parse Command Line Options

while [ $# -gt 0 ]; do
    case $1 in
        -h) usage 0;;
        esac
    shift
done

# Display Information

echo "Temperature: $(temperature) degrees"
```

- The general flow of your script should be to parse arguments and then to execute different pipelines.
- You will need to expand the `case` statement to parse command line arguments.
- You will need to use `if` statements to guard or control which could gets executed.
- You will probably want to use `curl` to fetch the weather information.
- You will probably want to use `sed` and `grep` and maybe `awk`.

- You will need to trim leading and trailing whitespace for the forecast.

Task 2: test_weather.sh

To aid you in testing the `weather.sh` script, we provided you with `test_weather.sh`, which you can use as follows:

```
$ ./test_weather.sh
Testing weather.sh ...
Usage ... Success
Default ... Success
46556 ... Success
46556 Celsius ... Success
46556 Forecast ... Success
46556 Celsius Forecast ... Success
92867 ... Success
92867 Celsius ... Success
92867 Forecast ... Success
92867 Celsius Forecast ... Success

Score 4.00
```

⌚ We'll Do It Live

Because the data is being pulled from a remote website, the tests might take a while (but no more than 30 seconds).

🔄 Retry the Pipeline

Sometimes the **pipeline fails** not because your code is wrong, but because of **hiccups with the server** (it is live data after all). If you suspect that your code is correct, you can always **retry** the pipeline to see if it will succeed without changing your code.

If it doesn't work after **three** retries... then your code is probably wrong.

Activity 2: Zipcode (4 Points)

Being able to lookup weather information based on zip codes is great... if you know the zip code of the place you are interested in. Like most people, however, Stella is not really familiar with zip codes around the country.

Like the instructor, Stella hails from Orange County¹, which is situated on the Best Coast and is the home of Mickey Mouse, John Wayne, No Doubt², and StarCraft³ (among other things). Having never really left **paradise**⁴, Stella doesn't really know much about other places in America. This is problematic as she has made many friends from all over the country, such as her roommate from Toledo, Ohio, or her other roommate from Jacksonville, Florida⁵. Since she needs to figure out the zip codes to these unfamiliar places, she is creating a script called `zipcode.sh`, which scrapes the zip codes from the website Zip Codes To Go and allows her to list all the zip codes in a specific state or even a particular city.

For instance, using `curl`, she can view all the raw HTML for Indiana by doing the following:

```
$ curl -s https://www.zipcodestogo.com/Indiana/
```

Because you are pretty good with regular expressions now, you decide to help Stella out with parsing this HTML and extracting the zip codes.

Task 1: zipcode.sh

The `zipcode.sh` script takes three possible flags:

```
$ ./zipcode.sh -h
Usage: zipcode.sh

  -c      CITY      Which city to search
  -s      STATE     Which state to search (Indiana)

If no CITY is specified, then all the zip codes for the STATE are displayed.
```

The `-c` flag takes a `CITY` argument, which specifies the city to search for within the `STATE`. If no `CITY` is specified, then the script should return all the zip codes in the `STATE`.

The `-s` flag takes a `STATE` argument, which specifies the `STATE` to search through. If no `STATE` is specified, then the script should assume the `STATE` is "Indiana".

The `-h` flag will display the usage message and exit with success.

Here are some examples of `zipcode.sh` in action:

```
# Show all Zip Codes from default state (Indiana)
$ ./zipcode.sh
46001
46011
46012
46013
...
47994
47995
47996
47997

# Show all Zip Codes in Notre Dame, Indiana
$ ./zipcode.sh -s Indiana -c "Notre Dame"
46556
```

Hints

Here is a skeleton you can use to start your `zipcode.sh` script:

```
#!/bin/sh

# Globals

URL=https://www.zipcodestogo.com/

# Functions

usage() {
}

# Parse Command Line Options

while [ $# -gt 0 ]; do
    case $1 in
        -h) usage 0;;
        *) usage 1;;
    esac
    shift
done

# Filter Pipeline(s)

curl -s $URL
```

- The general flow of your script should be to parse arguments and then to execute different pipelines.
- You will probably want to use sed and grep and maybe awk.
- You will probably want to create different functions and use them as stages in your pipelines.

Task 2: test_zipcode.sh

To aid you in testing the `zipcode.sh` script, we provided you with `test_zipcode.sh`, which you can use as follows:

```
$ ./test_zipcode.sh
Testing zipcode.sh ...
Usage ... Success
Default ... Success
Indiana ... Success
Notre Dame, Indiana ... Success
South Bend, Indiana ... Success
California ... Success
Orange, California ... Success
Los Angeles, California ... Success
New York ... Success
Buffalo, New York ... Success
New York, New York ... Success

Score 4.00
```

☹ We'll Do It Live

Because the data is being pulled from a remote website, the tests might take a while (but no more than 30 seconds).

Activity 3: Quiz (2 Points)

Once you have completed all the activities above, you are to complete the following **reflection quiz**:

- Which of the following statements regarding **pipelines** are true (select all that apply)?
 - ☐ Pipelines are the ultimate manifestation of the **Unix Philosophy**.
 - ☐ Pipelines are examples of **concurrency**.
 - ☐ Pipelines enable opportunities for **parallelism**.
 - ☐ Pipelines work by sending the standard output and error of one process to the standard input of another process.
 - ☐ Pipelines are an uncommon computational pattern only found on Unix systems.
- When using `curl`, we sometimes need to use the `-L` flag. Why is that?
 - ☐ This flag tells `curl` to filter any URLs present in the requested resource.
 - ☐ This flag tells `curl` to save the requested resource to the local file system.
 - ☐ This flag tells `curl` to follow any HTTP redirects in the requested resource.
 - ☐ This flag tells `curl` to listen on a specific port for the requested resource.
- When using `grep` or `sed`, we sometimes need to use the `-E` flag. Why is that?
 - ☐ This flag tells `grep` and `sed` to interpret patterns as **shell globs**.
 - ☐ This flag tells `grep` and `sed` to interpret patterns as **extended regular expressions**.
 - ☐ This flag tells `grep` and `sed` to interpret patterns as **literal strings** rather than regular expressions.
 - ☐ This flag tells `grep` and `sed` to interpret patterns in an **case-insensitive manner**.
- Given the list of words located at `/usr/share/dict/words` on the **student** machines (or <https://yld.me/raw/bf7U> online), use filters and pipelines to perform the following tasks:

1. **Count** the number of words that **end with the letter "y"**. ____

```
abacay
abactinally
abandonedly
abasedly
abashedly
...
```

2. Ignoring case, **count** the number of words that **start with the letter "b"**. ____

```
B
B-
b
B911
B.A.
...
```

3. Ignoring case, **count** the number of words that have **three consecutive vowels** (ie. a, e, i, o, u). ____


```
...
zygophyceae
zygophyceous
zygophyllaceae
zygophyllaceous
zygopteraceae
```

4. Ignoring case, **count** the number of words that **contain the pattern "vowel-character-character-vowel"**, where "vowel" and "character" refer to the same character respectively. ____

```
...
zocco
zoccolo
zoeller
zorillinae
zorro
```

5. **Count** the number of words **contain at least two digits, but don't end in a digit**. ____

```
10-point
10th
11-point
12-point
16-point
...
```

1.	
2.	
3.	
4.	
5.	

Generate

As with Reading 01, you will need to store your answers in a `homework03/answers.json` file. You can use the form above to generate the contents of this file, or you can write the JSON by hand.

To test your quiz, you can use the `test_quiz.py` script:

```
$ ./test_quiz.py
Submitting homework03 quiz ...
Q01 0.25
Q02 0.25
Q03 0.25
Q04 1.25

Score 2.00
```

Guru Point (1 Extra Credit Point)

Professor Tim Weninger, Notre Dame's resident Reddit expert, lambasted the instructor for failing to introduce students to cron in previous course sections. To rectify this, you are to create a cronjob that periodically uses your `weather.sh` script to output the weather for a zip code of your choice in a file in your `HOME` directory. You will then need to modify your `~/.bashrc` file to output the contents of the resulting weather file every time you login.

To help you setup your cronjob, here are some resources regarding:

- Scheduling Tasks with Cron Jobs
- How To Add Jobs To cron Under Linux or UNIX
- How To Schedule Routine Tasks With Cron and Anacron on a VPS

✓ Verification

To get credit for this Guru Point, show your cronjob in action to the instructor or a TA to verify. You have up until a week **after** this assignment is due to verify your Guru Point.

Self-Service Extension

Remember that you can always **forgo** this Guru Point for **two extra days** to do the homework. That is, if you need an extension, you can simply skip the Guru Point and you will automatically have until Monday to complete the assignment for full credit.

Just leave a note on your **Merge Request** of your intensions.

Submission

To submit your assignment, please commit your work to the `homework03` folder of your `homework03` branch in your **assignments** GitLab repository:

```
#-----
# BE SURE TO DO THE PREPARATION STEPS IN ACTIVITY 0
#-----

$ cd homework03                                # Go to Homework 03 directory

$ $EDITOR weather.sh                           # Edit script
$ git add weather.sh                           # Mark changes for commit
$ git commit -m "homework03: Activity 1 completed" # Record changes
...

$ $EDITOR zipcode.sh                           # Edit script
$ git add zipcode.sh                           # Mark changes for commit
$ git commit -m "homework03: Activity 2 completed" # Record changes
...

$ $EDITOR answers.json                         # Edit quiz
$ git add answers.json                         # Mark changes for commit
$ git commit -m "homework03: Activity 3 completed" # Record changes
...

$ git push -u origin homework03                # Push branch to GitLab
```

Merge Request

Remember to create a merge request and assign the appropriate TA from the Reading 03 TA List.

DO NOT MERGE your own merge request. The TAs use open merge requests to keep track of which assignments to grade. Closing them yourself will cause a delay in grading and confuse the TAs.

1. Don't you dare call it the OC. 
2. Don't speak, I know what you're thinking; I don't need your reasons; Don't tell me cause it hurts. 
3. 1-A to victory. Zerg OP. 
4. Especially after last week... RIP. 
5. This for you WenTao and Pat. 210 Dillon Hall 4 Lyfe. 